

Slang Translation Model

REPORT

ON

Minor Project (ETMN100)

Submitted as a partial fulfilment toward the

award of Bachelor of Technology

in COMPUTER SCIENCE AND ENGINEERING

Submitted by

AYUSH CHAUDHARY

(A411052222008)

(2022 – 2026)

Under the Supervision of

Prof. Meenakshi Deshwal



Amity School of Engineering & Technology

AMITY UNIVERSITY, UTTAR PRADESH GREATER NOIDA

Certificate

I hereby certify that the work which is being offered in entitled Minor Project Report, in partial fulfilment of the requirement for the award of B. Tech in (CSE) and submitted to the Department of ASET is a genuine file of my very own and executed all through a duration from July-September 2024 under the supervision of Prof. Meenakshi Deshwal, Amity University Greater Noida, Computer Science Department.

The matter provided on this undertaking record has not been submitted via me for the award of another degree someplace else.

AYUSH CHAUDHARY
(A41105222008)

This is certified that the above statement made with the aid of the candidate is accurate to the exceptional of my expertise.

Date:01/10/2025

Signature of Faculty Guide
MS. MEENAKSHI

Abstract

The translation of formal English into contemporary informal language and slang presents a significant challenge for natural language processing, as it requires an understanding of cultural context, nuance, and non-literal expressions. This project addresses this challenge by developing a sophisticated linguistic style transfer model capable of performing this task. The primary objective was to fine-tune a powerful, pre-trained Large Language Model (LLM) to accurately capture the subtleties of informal communication.

The methodology centers on the Mistral 7B model, a high-performing yet efficient transformer architecture. To make the fine-tuning process computationally feasible, the project employed Quantized Low-Rank Adaptation (QLoRA), a parameter-efficient fine-tuning technique. A parallel corpus of formal and informal phrases was sourced from Kaggle and subjected to a rigorous preprocessing pipeline, which included normalization, contraction expansion, and formatting into an instruction-based structure suitable for training.

The model was trained over multiple epochs, with results demonstrating a significant improvement in translation quality compared to a baseline model. Evaluation was conducted using both qualitative analysis and advanced semantic similarity metrics like COMET, which confirmed the final model's superior ability to generate contextually appropriate and natural-sounding slang. This project successfully demonstrates that fine-tuning LLMs with efficient techniques like QLoRA is a highly effective approach for complex and nuanced text style transfer tasks.

Acknowledgements

I would love to express my sincerest gratitude and indebtedness to the person that gave me ethical and technical help & whose kind assistance has been instrumental inside the crowning glory of my project. It offers me sizeable satisfaction to very own my humble gratitude to my faculty guide **Ms. Meenakshi Deshwal** for this integral steering and for supplying necessary thoughts to perform this challenge.

I would, then the like to vicinity on record my best regards and private sense of gratitude to **Prof. (Dr) AJAY RANA (Director General)**, **Prof. (Dr) Deepshikha Bhargava (HOI, CSE/IT/ECE/EEE)** and all the professors of our Computer Science Department. I additionally thank all the faculties of Amity University for his or her cautious and precious steerage which has been extraordinarily treasured for my look at both theoretically and almost.

I understand this possibility as a huge milestone in my professional improvement. I will try to apply abilities and understanding won in the course of this era in a great feasible manner, and I will retain paintings, to obtain my preferred professional targets.

Sincerely

AYUSH CHAUDHARY

(A41105222008)

Computer Science & Engineering Department, (ASET)

Declaration

I, Ayush Chaudhary, hereby declare that the work presented in this project report, entitled "The Slang Translator Project," is a record of my own original research and investigation. The work embodied in this report has not been submitted in whole or in part for any other degree or diploma at this or any other university.

All information and concepts derived from external sources, whether published or unpublished, have been duly acknowledged and referenced in the text.

SignedXXX XXX..... Date28/09/2025.....

Contents

Certificate	i
Abstract	ii
Acknowledgements	iii
Declaration	iv
1 Introduction	1
1.1 Background	1
1.2 About this Thesis	3
1.3 Chapter List	4
2 Background Research	5
3 Design	7
4 Implementation	11
5 Evaluation & Testing	21
6 Conclusion	24
6.1 Conclusions	24
6.2 Future Work	25
A Project Specification	26
A.1 Functional Requirements	27
A.2 Non-Functional Requirements	28
B Project Management	31
C Project Demo	34
D References	35

List of Tables

5.1	TABLE 5.1. QUANTITATIVE PERFORMANCE METRICS	23
5.2	TABLE 5.2. QUALITATIVE ANALYSIS OF MODEL OUTPUTS	23

List of Figures

3.1	First subfigure	8
3.2	Second subfigure	8
3.3	Third subfigure	8
3.4	System Architecture	8
3.5	Data Preprocessing	9
3.6	QLoRA Fine Tuning	9
4.1	Initial data loading and inspection.	13
4.2	Applying text normalization and cleaning.	13
4.3	Applying text normalization and cleaning	14
4.4	Fine tuning	15
4.5	Fine Tuning	16
4.6	Fine Tuning	17
4.7	Fine Tuning	17
4.8	Inference	18
4.9	Inference	19
4.10	Inference	20
5.1	Model Performance Comparison	21
5.2	Training Loss Curve	22
C.1	Model's output	34

List of Algorithms

Core Model and Learning Algorithms

Transformer Architecture: This is the foundational blueprint of the Mistral 7B model. Its core algorithmic innovation is the Self-Attention mechanism, which allows the model to weigh the importance of different words in the input text to better understand context.

Backpropagation: This is the fundamental algorithm used during training. It calculates the error (or "loss") in the model's predictions and propagates this error backward through the network to update the trainable weights.

Paged AdamW Optimizer: This is the specific optimization algorithm used to perform the weight updates during training. It is a memory-efficient variant of the Adam optimizer, designed to handle large models by "paging" optimizer states between the CPU and GPU.

Parameter-Efficient Fine-Tuning (PEFT) Algorithms

Quantization (4-bit NormalFloat): This algorithm reduces the memory footprint of the model by compressing its weights from a higher precision (like 16-bit) down to a 4-bit representation.

LoRA (Low-Rank Adaptation): This is the core fine-tuning algorithm. Instead of training all the model's billions of parameters, LoRA freezes them and trains a much smaller set of "adapter" matrices, which are then used to modify the model's behavior.

QLoRA: This is the main technique used in the project, which is an algorithm that combines Quantization and LoRA. It allows for the efficient fine-tuning of the compressed 4-bit model.

Data Processing and Generation Algorithms

Regular Expressions (Regex): This algorithm was used in the preprocess_data.py script for pattern-based text cleaning, such as finding and normalizing elongated words (e.g., "soooo" to "soo").

Sampling: This is the decoding algorithm used during inference (infer.py) to generate text. Instead of

just picking the single most likely next word (greedy decoding), sampling selects the next word from a probability distribution, which allows for more creative and less repetitive outputs.

Chapter 1

Introduction

The increasing prevalence of informal language and slang in digital communication highlights a significant challenge for Natural Language Processing: effectively translating formal English into its nuanced, context-dependent informal equivalents. Unlike traditional language translation, this "style transfer" task requires understanding underlying intent and rephrasing according to dynamic cultural norms, a capability often beyond conventional models. This project aims to address this by developing a slang translator through the fine-tuning of a state-of-the-art Large Language Model (LLM), Mistral 7B Instruct. Utilizing the memory-efficient Quantized Low-Rank Adaptation (QLoRA) technique, we demonstrate how a powerful LLM can be adapted to this complex linguistic style transfer task using a parallel corpus of formal and informal phrases. This report details the methodology, from data preprocessing to QLoRA-based training, and presents an evaluation of the model's performance, showcasing its ability to generate natural and contextually appropriate slang translations.

1.1 Background

The task of translating formal English into informal slang is a complex challenge that lies at the intersection of machine translation, natural language understanding, and computational linguistics. To fully appreciate the methodology employed in this project, it is essential to understand the technological evolution that has made such a task feasible. This section provides a background on the progression from traditional machine translation to the current paradigm of fine-tuning large language models.

From Machine Translation to Style Transfer

Machine Translation (MT) has been a long-standing goal of computer science. Early systems were dominated by Statistical Machine Translation (SMT), which used statistical models learned from large bilingual text corpora. While revolutionary for their time, SMT systems struggled with grammatical fluency and capturing the broader context of a sentence.

The paradigm shifted with the advent of Neural Machine Translation (NMT), which utilized deep learning models, particularly Recurrent Neural Networks (RNNs) and their variants like Long Short-Term

Memory (LSTM) networks. These models could process language sequentially, allowing them to learn grammatical structures and long-range dependencies more effectively, leading to significantly more fluent and accurate translations.

The project at hand, however, is not a traditional translation task but one of Text Style Transfer (TST). The goal of TST is to modify the stylistic attributes of a text (e.g., formality, sentiment, tone) while preserving its core semantic content. This is inherently more challenging than translating between languages because style is often subjective and deeply tied to cultural and contextual nuance. The challenge is not just to find equivalent words but to rephrase an idea entirely in a different linguistic register.

The Transformer Architecture: A Paradigm Shift

The most significant breakthrough in modern Natural Language Processing (NLP) was the introduction of the Transformer architecture in the 2017 paper "Attention Is All You Need" by Vaswani et al. The Transformer model dispensed with the sequential nature of RNNs and introduced the concept of the self-attention mechanism.

Self-attention allows the model to weigh the importance of all other words in the input sequence when processing any given word. In essence, it enables the model to build a rich, context-aware representation of each word by understanding its relationship with every other word in the text. This parallel processing capability, combined with its effectiveness in capturing long-range dependencies, made the Transformer vastly more powerful and efficient than its predecessors. This architecture is now the foundational building block for virtually all state-of-the-art large language models.

The Rise of Large Language Models (LLMs)

The success of the Transformer architecture led to the creation of Large Language Models (LLMs) like Google's BERT and OpenAI's GPT series. These models are pre-trained on enormous, web-scale datasets containing trillions of words. During this pre-training phase, the model learns a general, deep understanding of language, including grammar, facts, reasoning abilities, and different linguistic styles.

This led to the dominant pre-training and fine-tuning paradigm. Instead of training a model from scratch for a specific task, researchers and developers can take a pre-trained LLM and fine-tune it on a much smaller, task-specific dataset. This process of transfer learning adapts the model's general knowledge to the specific requirements of the new task. Our project is a direct application of this paradigm, adapting a general-purpose LLM to the highly specific task of slang translation.

Parameter-Efficient Fine-Tuning (PEFT): The Key to Accessibility

While incredibly powerful, LLMs contain billions of parameters, and fine-tuning all of them (a process known as full fine-tuning) is computationally prohibitive, requiring massive amounts of GPU memory and time. To overcome this barrier, Parameter-Efficient Fine-Tuning (PEFT) methods were developed.

The most prominent of these is Low-Rank Adaptation (LoRA). LoRA freezes the billions of weights in the pre-trained model and injects small, trainable "adapter" layers into the architecture. These adapters

consist of low-rank matrices that approximate the weight updates that would occur during full fine-tuning. By training only these adapters—which represent a tiny fraction (often ≤ 1

Building on this, Quantized Low-Rank Adaptation (QLoRA) further democratized the process. QLoRA first quantizes the pre-trained model, reducing the precision of its weights from 16-bit or 32-bit to an efficient 4-bit representation. This drastically cuts down the memory required to load the model. Then, it uses LoRA to fine-tune this quantized model. The combination of quantization and LoRA allows for the fine-tuning of massive models on a single, consumer-grade GPU, a technique that is central to the methodology of this project.

1.2 About this Thesis

This is the thesis of *Ayush Chaudhary*, submitted as part of the requirements for the degree of B.Tech Computer Science and Engineering at the Amity University, Greater Noida.

This thesis posits that the fine-tuning of a pre-trained large language model (LLM) using parameter-efficient techniques presents a highly effective and accessible solution to the complex challenge of formal-to-informal linguistic style transfer. The central expectation of this work is to demonstrate that a modern LLM, specifically Mistral 7B, can be successfully adapted to comprehend and generate nuanced, culturally-specific informal language and slang, a task that has traditionally been a significant hurdle for automated systems. We hypothesize that by leveraging an instruction-based training format on a curated parallel corpus, the model will move beyond simple lexical substitution and learn the deeper syntactic and stylistic patterns that define informal communication.

A primary expectation of this project is to validate the efficacy of the Quantized Low-Rank Adaptation (QLoRA) methodology. We anticipate that this technique will enable the successful fine-tuning of a multi-billion parameter model within a resource-constrained environment, proving it to be a viable strategy for specialized NLP research and development without the need for large-scale industrial infrastructure. The expectation is that despite training less than 1% of the model's total parameters, the resulting "adapter" weights will be sufficient to imbue the model with the specialized skill of slang translation.

Furthermore, we expect to see a clear and measurable improvement in performance correlated with the duration of training. It is anticipated that a model trained for multiple epochs will significantly outperform a baseline model trained for only a single epoch. This superiority is expected to manifest not only in qualitative assessments, where the translations will appear more natural and contextually appropriate to a human evaluator, but also in quantitative terms, particularly through advanced semantic metrics like COMET. The final expectation is that this project will serve as a successful proof-of-concept, providing a robust and reproducible framework for creating specialized style transfer models for other niche linguistic domains.

1.3 Chapter List

Provide a list of all the chapters within the thesis and a brief summary of the content.

Chapter 2 Background Research.

Chapter 3 Design.

Chapter 4 Implementation.

Chapter 5 Evaluation & Testing.

Chapter 6 Conclusion. The conclusions of the thesis are presented.

Chapter 2

Background Research

The task of translating formal English into informal slang is a complex challenge that lies at the intersection of machine translation, natural language understanding, and computational linguistics. To fully appreciate the methodology employed in this project, it is essential to understand the technological evolution that has made such a task feasible. This chapter provides a background on the progression from traditional machine translation to the current paradigm of fine-tuning large language models.

2.1 From Machine Translation to Style Transfer

Machine Translation (MT) has been a long-standing goal of computer science. Early systems were dominated by Statistical Machine Translation (SMT), which used statistical models learned from large bilingual text corpora. While revolutionary for their time, SMT systems struggled with grammatical fluency and capturing the broader context of a sentence.

The paradigm shifted with the advent of Neural Machine Translation (NMT), which utilized deep learning models, particularly Recurrent Neural Networks (RNNs) and their variants like Long Short-Term Memory (LSTM) networks. These models could process language sequentially, allowing them to learn grammatical structures and long-range dependencies more effectively, leading to significantly more fluent and accurate translations.

The project at hand, however, is not a traditional translation task but one of Text Style Transfer (TST). The goal of TST is to modify the stylistic attributes of a text (e.g., formality, sentiment, tone) while preserving its core semantic content. This is inherently more challenging than translating between languages because style is often subjective and deeply tied to cultural and contextual nuance. The challenge is not just to find equivalent words but to rephrase an idea entirely in a different linguistic register.

2.2 The Transformer Architecture: A Paradigm Shift

The most significant breakthrough in modern Natural Language Processing (NLP) was the introduction of the Transformer architecture in the 2017 paper "Attention Is All You Need" by Vaswani et al.. The Transformer model dispensed with the sequential nature of RNNs and introduced the concept of the

self-attention mechanism.

Self-attention allows the model to weigh the importance of all other words in the input sequence when processing any given word. In essence, it enables the model to build a rich, context-aware representation of each word by understanding its relationship with every other word in the text. This parallel processing capability, combined with its effectiveness in capturing long-range dependencies, made the Transformer vastly more powerful and efficient than its predecessors. This architecture is now the foundational building block for virtually all state-of-the-art large language models.

2.3 The Rise of Large Language Models (LLMs)

The success of the Transformer architecture led to the creation of Large Language Models (LLMs) like Google’s BERT and OpenAI’s GPT series. These models are pre-trained on enormous, web-scale datasets containing trillions of words. During this pre-training phase, the model learns a general, deep understanding of language, including grammar, facts, reasoning abilities, and different linguistic styles.

This led to the dominant pre-training and fine-tuning paradigm. Instead of training a model from scratch for a specific task, researchers and developers can take a pre-trained LLM and fine-tune it on a much smaller, task-specific dataset. This process of transfer learning adapts the model’s general knowledge to the specific requirements of the new task. Our project is a direct application of this paradigm, adapting a general-purpose LLM to the highly specific task of slang translation.

2.4 Parameter-Efficient Fine-Tuning (PEFT): The Key to Accessibility

While incredibly powerful, LLMs contain billions of parameters, and fine-tuning all of them (a process known as full fine-tuning) is computationally prohibitive, requiring massive amounts of GPU memory and time. To overcome this barrier, Parameter-Efficient Fine-Tuning (PEFT) methods were developed.

The most prominent of these is Low-Rank Adaptation (LoRA). LoRA freezes the billions of weights in the pre-trained model and injects small, trainable “adapter” layers into the architecture. These adapters consist of low-rank matrices that approximate the weight updates that would occur during full fine-tuning. By training only these adapters—which represent a tiny fraction (often $\leq 1\%$) of the total parameters—LoRA makes fine-tuning accessible without sacrificing significant performance.

Building on this, Quantized Low-Rank Adaptation (QLoRA) further democratized the process. QLoRA first quantizes the pre-trained model, reducing the precision of its weights from 16-bit or 32-bit to an efficient 4-bit representation. This drastically cuts down the memory required to load the model. Then, it uses LoRA to fine-tune this quantized model. The combination of quantization and LoRA allows for the fine-tuning of massive models on a single, consumer-grade GPU, a technique that is central to the methodology of this project.

Chapter 3

Design

The design of the Slang Translator Project follows a systematic, modular pipeline architecture. Each stage of the process, from data handling to model inference, is designed as a distinct module with specific responsibilities. This approach ensures clarity, maintainability, and allows for individual components to be tested and improved independently. This chapter details the design of the overall system architecture and its constituent modules.

3.1 System Architecture Overview

The architecture consists of four primary stages: Data Ingestion and Preparation, Model Fine-Tuning, Model Evaluation, and Inference. The output of each stage serves as the direct input for the next, creating a cohesive workflow from raw data to a functional translation model.

3.2 Data Design

The design of the data pipeline was critical to ensure the model received clean, consistent, and correctly formatted information.

Data Sourcing and Schema: The system was designed to use a parallel corpus. The "Formal & Informal Phrases" dataset was selected due to its direct applicability, containing paired sentences in the required styles. The initial data schema was designed as a simple CSV file (`raw_data.csv`) with two columns: `formal_text` and `informal_text`, providing a clear and easy-to-parse input structure.

Instruction Prompt Design: For the fine-tuning stage, the data was designed to be transformed into an instruction-based format. The prompt structure "Translate the following formal English sentence to informal slang: "formal_text" "informal_text" was deliberately chosen. This design directly aligns with the training methodology of the Mistral 7B Instruct model, which is pre-trained to follow commands enclosed within [INST] tags. This ensures the model clearly understands its task during fine-tuning, leading to more effective learning. The final dataset was designed as a JSONL file (`formatted_dataset.jsonl`), where each line is a JSON object containing a single formatted prompt string.

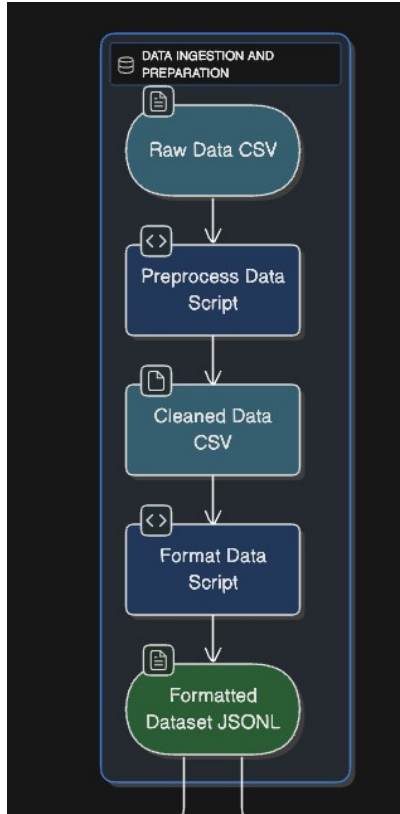


Figure 3.1: First subfigure

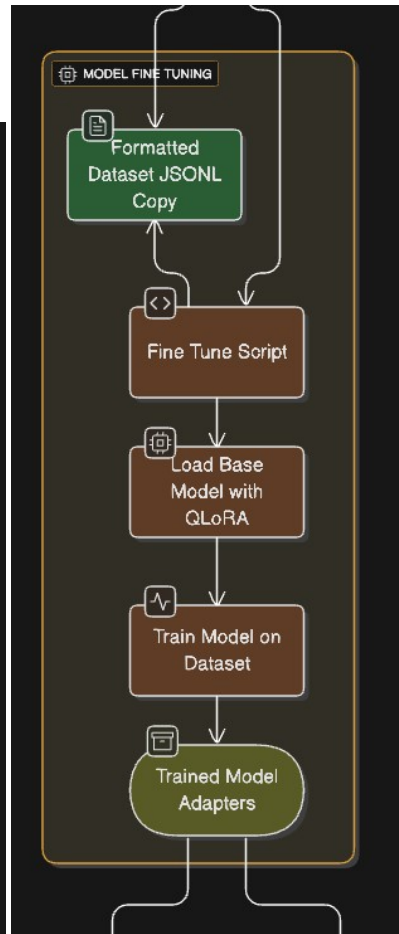


Figure 3.2: Second subfigure

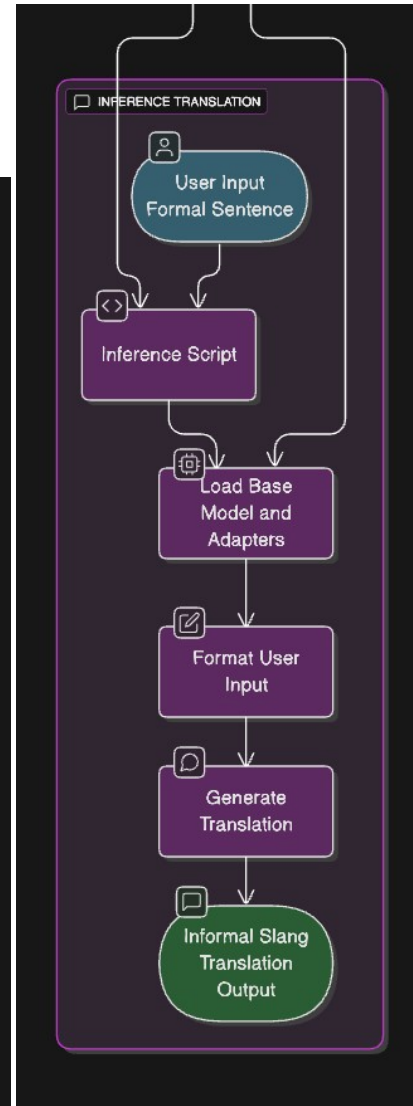


Figure 3.3: Third subfigure

Figure 3.4: System Architecture

3.3 Preprocessing Module Design

The preprocessing module (`preprocess_data.py`) was designed to be a self-contained script that executes a series of cleaning functions in a specific order. Each function was designed to address a common issue in raw text data:

Case Normalization: Designed to convert all text to lowercase, reducing the vocabulary size and ensuring the model does not treat the same word with different capitalization as separate tokens.

Contraction Expansion: Designed to expand contractions (e.g., "I'm" to "I am") to standardize the grammatical structure of the input text.

Punctuation Removal: Designed to eliminate punctuation, which can act as noise and is often used inconsistently in informal text.

Emoji Handling: Designed to convert emojis into textual descriptions (e.g., "red-heart-emoji" to

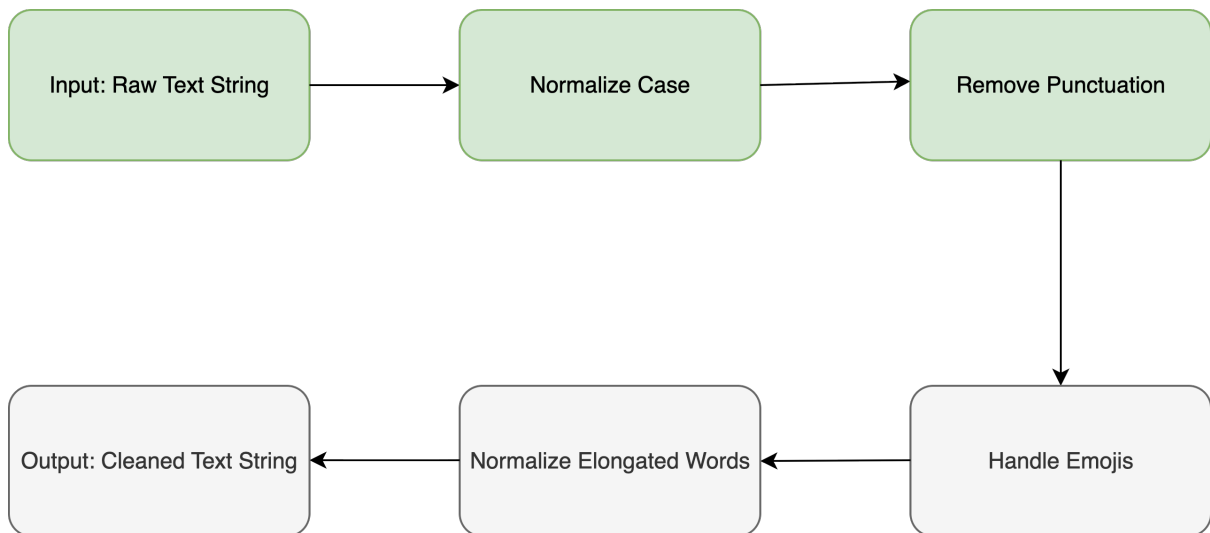


Figure 3.5: Data Preprocessing

(`red_heart`) to preserve their semantic meaning in a way the model can process.

Elongated Word Normalization: Designed to handle expressive lengthening of words (e.g., "soooo") by normalizing them, which helps in reducing vocabulary sparsity.

3.4 Model and Training Design

The core of the project is the model and the strategy used to train it.

Base Model Selection: The Mistral 7B Instruct model was selected by design. The key design considerations were its high performance on benchmark tasks relative to its size and, crucially, its instruction-following capabilities. This made it an ideal candidate for a task that can be explicitly commanded.

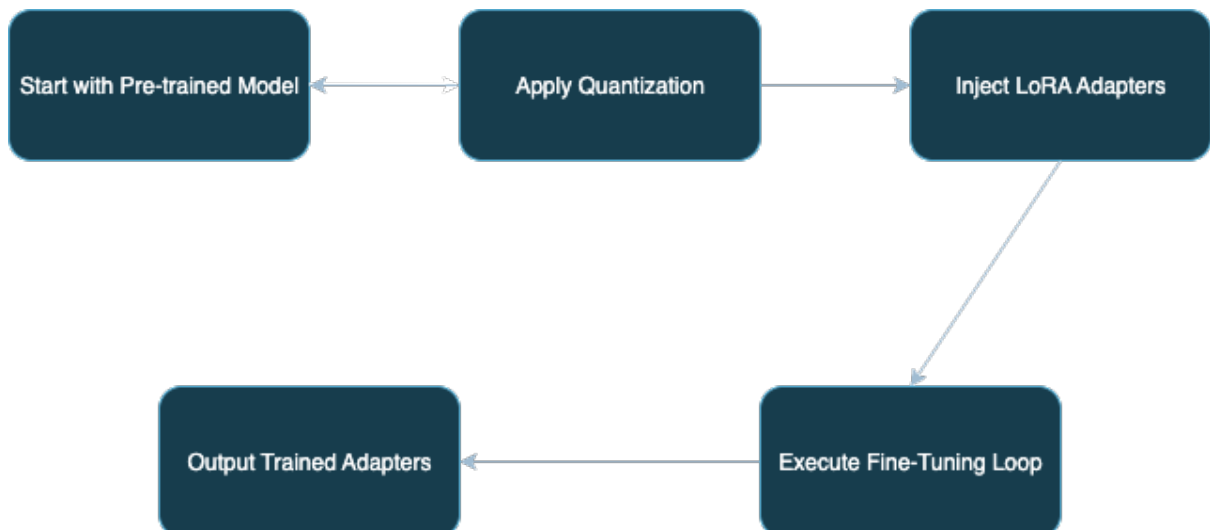


Figure 3.6: QLoRA Fine Tuning

Fine-Tuning Strategy: The QLoRA technique was the chosen design for fine-tuning. This decision was driven by hardware and resource constraints. By quantizing the base model to 4-bits and training only

small, low-rank adapters, this design made it feasible to fine-tune a 7-billion parameter model on a single consumer-grade GPU. This parameter-efficient approach was a core design choice to ensure the project's feasibility.

3.5 Inference Module Design

The inference module (`infer.py`) was designed to load the trained model adapters and perform translations on new, unseen formal text. Its design includes loading the base model in its quantized form and then merging the trained LoRA adapter weights for inference. A separate, CPU-compatible version of this module was also designed by removing the quantization step. This dual design ensures that the model can be run for high-performance inference on a GPU, but also allows for accessibility and testing on machines without a dedicated GPU, albeit with a trade-off in speed.

Chapter 4

Implementation

The implementation phase translated the architectural design into a functional system. This chapter details the development environment, the key libraries utilized, and the specific implementation of each module in the project pipeline, from data preparation to model inference.

4.1 Development Environment and Tools

The project was developed using a modern, standardized Python-based environment to ensure reproducibility and efficiency.

Integrated Development Environment (IDE): The primary IDE used was Cursor, a fork of Visual Studio Code, which provides a robust environment for coding, debugging, and terminal access.

Programming Language: The project was implemented entirely in Python 3.9.

Dependency Management: To maintain a clean and isolated environment, project dependencies were managed using Python's built-in venv module. All required libraries were listed in a requirements.txt file, allowing for easy one-step installation using pip.

Core Libraries: The implementation heavily relies on the Hugging Face ecosystem and other key data science libraries:

PyTorch: Served as the core deep learning framework for tensor operations and GPU acceleration.

Hugging Face Transformers: Used to load the pre-trained Mistral 7B model and its tokenizer from the Hugging Face Hub.

Hugging Face PEFT (Parameter-Efficient Fine-Tuning): The key library for implementing the QLoRA technique. It was used to configure the LoraConfig and apply the adapter layers to the base model.

Hugging Face TRL (Transformer Reinforcement Learning): Provided the SFTTrainer, a high-level utility that simplifies the supervised fine-tuning process by handling the training loop, data collation, and optimization.

bitsandbytes: The backend library that enabled the 4-bit quantization of the base model.

Pandas: Used for all data manipulation tasks, including reading the initial CSV dataset and writing the processed data back to a new file.

4.2 Data Preprocessing Implementation

The data preprocessing module (`preprocess_data.py`) was implemented as a Python script that reads the `raw_data.csv` file, applies a series of cleaning functions, and saves the output. The implementation used the *pandas* library to manage the data in a `DataFrame` structure. Key functions were implemented using the *re* (regular expressions) library for pattern-based text normalization and the *emoji* library to convert emojis to their textual representations.

4.3 Model Fine-Tuning Implementation

This module (`fine_tune.py`) represents the core of the project. The implementation involved three key steps:

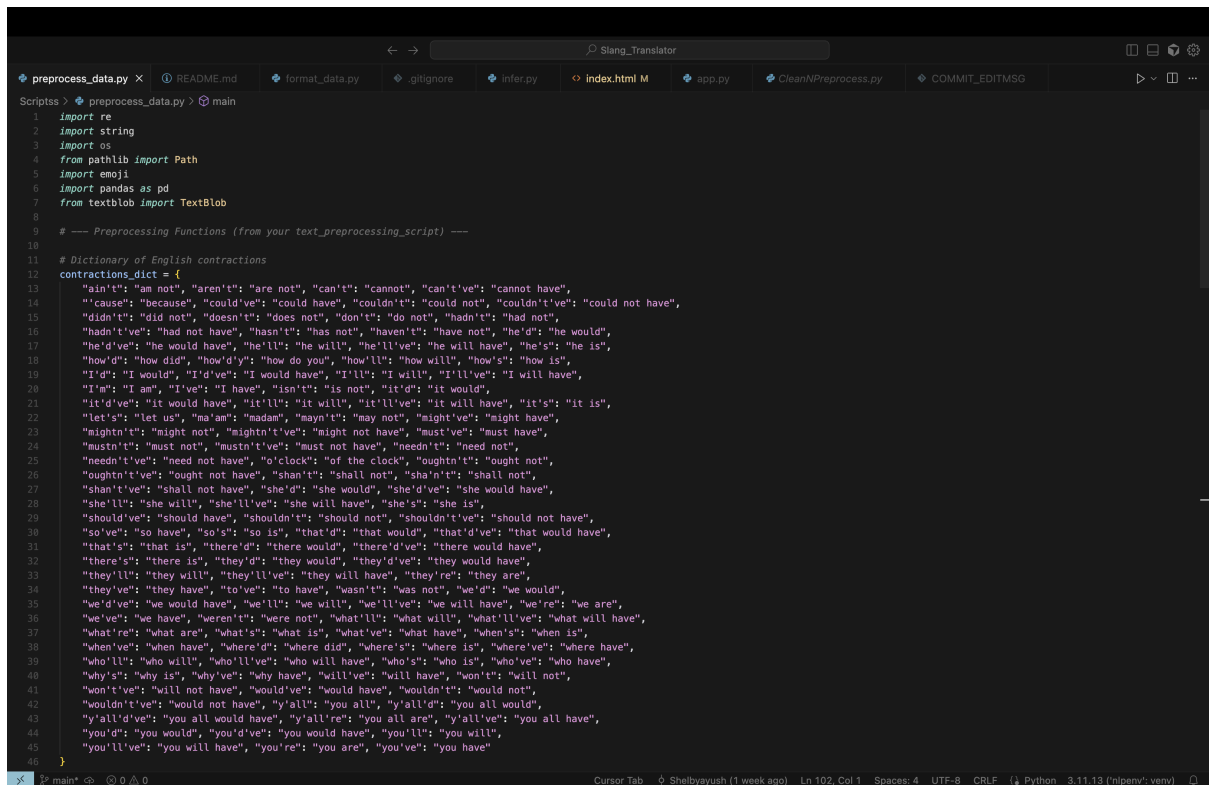
Model and Quantization Setup: The `BitsAndBytesConfig` class from the *transformers* library was configured to load the Mistral 7B model in 4-bit precision. This was passed directly to the `from_pretrained` method.

QLoRA Configuration: The `LoraConfig` class from the *PEFT* library was instantiated to define the parameters of the LoRA adapters, such as the rank (`r`), `lora_alpha`, and the target modules within the model's architecture. The `get_peft_model` function was then used to wrap the base model with these trainable adapters.

Trainer Initialization: The `SFTTrainer` from the *TRL* library was used to orchestrate the training. It was initialized with the PEFT-configured model, the formatted dataset, the tokenizer, and a `TrainingArguments` object that specified hyperparameters like the learning rate, number of epochs, and batch size. The entire training process was initiated by a single call to `trainer.train()`.

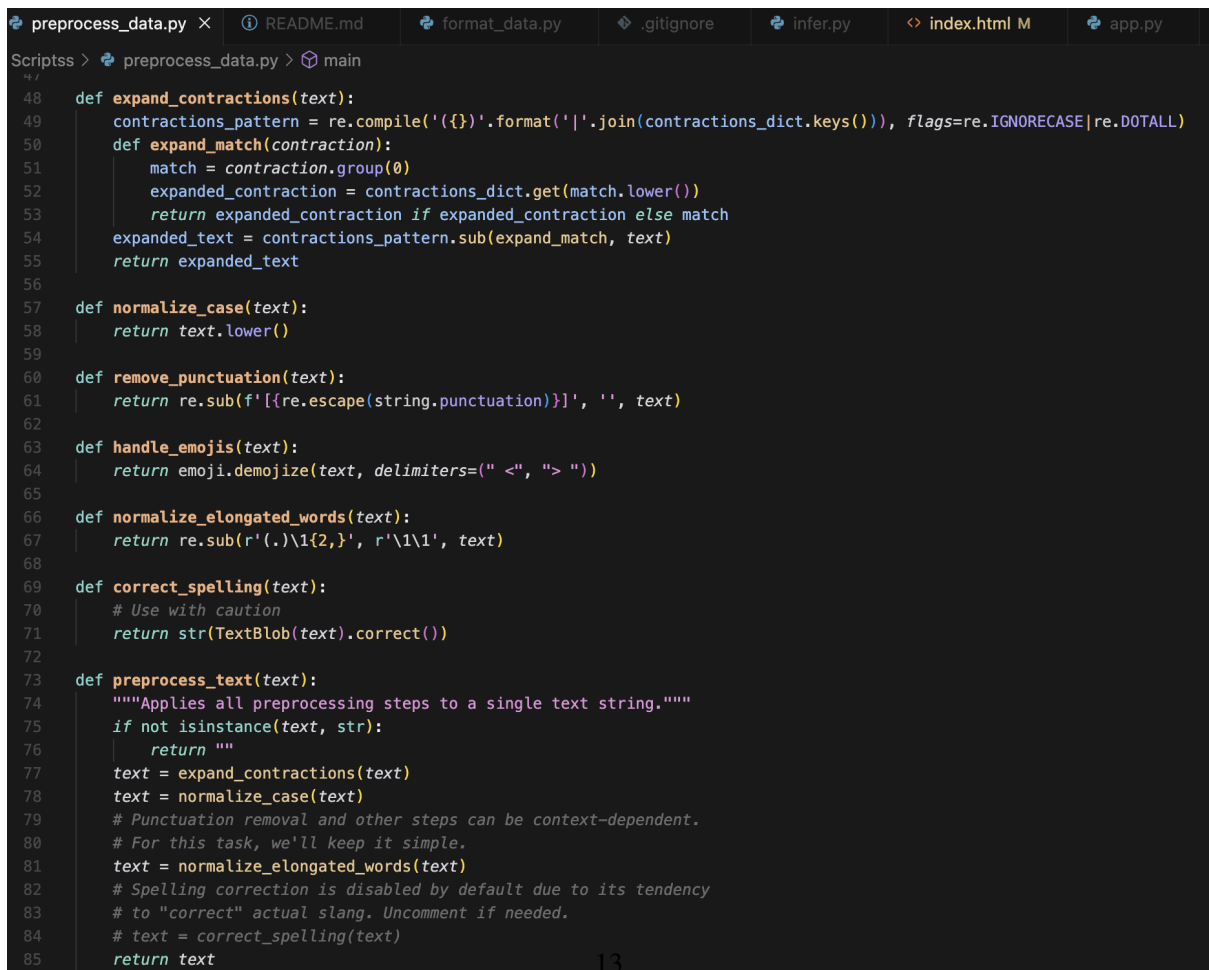
4.4 Inference Implementation

The final inference module (`infer.py`) was implemented to use the trained model. The script loads the base Mistral 7B model and tokenizer. Crucially, it then uses the `PeftModel.from_pretrained` function to load the trained adapter weights from the checkpoint directory and merge them with the base model. The translation is performed by calling the `model.generate()` function with a tokenized input prompt. The implementation also includes a final step to parse the raw output from the model, extracting only the clean, translated text.



```
1 import re
2 import string
3 import os
4 from pathlib import Path
5 import emoji
6 import pandas as pd
7 from textblob import TextBlob
8
9 # --- Preprocessing Functions (from your text_preprocessing_script) ---
10
11 # Dictionary of English contractions
12 contractions_dict = {
13     "ain't": "am not", "aren't": "are not", "can't": "cannot", "can't've": "cannot have",
14     "cause": "because", "could've": "could have", "couldn't": "could not", "couldn't've": "could not have",
15     "didn't": "did not", "doesn't": "does not", "don't": "do not", "hadn't": "had not",
16     "hadn't've": "had not have", "hasn't": "has not", "haven't": "have not", "he'd": "he would",
17     "he'd've": "he would have", "he'll": "he will", "he'll've": "he will have", "he's": "he is",
18     "how'd": "how did", "how'd've": "how do you", "how'll": "how will", "how's": "how is",
19     "I'd": "I would", "I'd've": "I would have", "I'll": "I will", "I'll've": "I will have",
20     "I'm": "I am", "I've": "I have", "isn't": "is not", "it'd": "it would",
21     "it'd've": "it would have", "it'll": "it will", "it'll've": "it will have", "it's": "it is",
22     "let's": "let us", "ma'am": "madam", "mayn't": "may not", "might've": "might have",
23     "mightn't": "might not", "mightn't've": "might not have", "must've": "must have",
24     "mustn't": "must not", "mustn't've": "must not have", "needn't": "need not",
25     "needn't've": "need not have", "o'clock": "of the clock", "oughtn't": "ought not",
26     "oughtn't've": "ought not have", "shan't": "shall not", "shan't've": "shall not",
27     "she'd": "she would", "she'd've": "she would have", "she'll": "she will", "she'll've": "she will have", "she's": "she is",
28     "should've": "should have", "shouldn't": "should not", "shouldn't've": "should not have",
29     "so've": "so have", "so's": "so is", "that'd": "that would", "that'd've": "that would have",
30     "that's": "that is", "there'd": "there would", "there'd've": "there would have",
31     "there's": "there is", "they'd": "they would", "they'd've": "they would have",
32     "they'll": "they will", "they'll've": "they will have", "they're": "they are",
33     "they've": "they have", "to've": "to have", "wasn't": "was not", "we'd": "we would",
34     "we'd've": "we would have", "we'll": "we will", "we'll've": "we will have", "we're": "we are",
35     "we've": "we have", "weren't": "were not", "what'll": "what will", "what'll've": "what will have",
36     "what're": "what are", "what's": "what is", "what've": "what have", "when's": "when is",
37     "when've": "when have", "where'd": "where did", "where's": "where is", "where've": "where have",
38     "who'll": "who will", "who'll've": "who will have", "who's": "who is", "who've": "who have",
39     "why's": "why is", "why've": "why have", "will've": "will have", "won't": "will not",
40     "won't've": "will not have", "would've": "would have", "wouldn't": "would not",
41     "wouldn't've": "would not have", "y'all": "you all", "y'all'd": "you all would",
42     "y'all'd've": "you all would have", "y'all're": "you all are", "y'all've": "you all have",
43     "you'd": "you would", "you'd've": "you would have", "you'll": "you will",
44     "you'll've": "you will have", "you're": "you are", "you've": "you have"
45 }
```

Figure 4.1: Initial data loading and inspection.



```
48 def expand_contractions(text):
49     contractions_pattern = re.compile('{{}}'.format('|'.join(contractions_dict.keys())) , flags=re.IGNORECASE|re.DOTALL)
50     def expand_match(contraction):
51         match = contraction.group(0)
52         expanded_contraction = contractions_dict.get(match.lower())
53         return expanded_contraction if expanded_contraction else match
54     expanded_text = contractions_pattern.sub(expand_match, text)
55     return expanded_text
56
57 def normalize_case(text):
58     return text.lower()
59
60 def remove_punctuation(text):
61     return re.sub(f'[{re.escape(string.punctuation)}]', '', text)
62
63 def handle_emojis(text):
64     return emoji.demojize(text, delimiters=(" <", "> "))
65
66 def normalize_elongated_words(text):
67     return re.sub(r'(\.){1,2}', r'\1', text)
68
69 def correct_spelling(text):
70     # Use with caution
71     return str(TextBlob(text).correct())
72
73 def preprocess_text(text):
74     """Applies all preprocessing steps to a single text string."""
75     if not isinstance(text, str):
76         return ""
77     text = expand_contractions(text)
78     text = normalize_case(text)
79     # Punctuation removal and other steps can be context-dependent.
80     # For this task, we'll keep it simple.
81     text = normalize_elongated_words(text)
82     # Spelling correction is disabled by default due to its tendency
83     # to "correct" actual slang. Uncomment if needed.
84     # text = correct_spelling(text)
85     return text
```

Figure 4.2: Applying text normalization and cleaning.

```

# --- Main Script Logic ---
def main():
    """Reads raw data, preprocesses it, and saves the cleaned data."""
    # Define file paths
    data_dir = Path("Dataa")
    raw_data_path = data_dir / "raw_data_fixed.csv"
    cleaned_data_path = data_dir / "cleaned_data.csv"

    # Check if raw data file exists
    if not raw_data_path.is_file():
        print(f"Error: Raw data file not found at {raw_data_path}")
        print("Please create the file and add data before running.")
        return

    print(f"Reading raw data from {raw_data_path}...")
    df = pd.read_csv(raw_data_path)

    print("Preprocessing data...")
    # Apply the preprocessing pipeline to both columns
    df['formal_text_cleaned'] = df['formal_text'].apply(preprocess_text)
    df['informal_text_cleaned'] = df['informal_text'].apply(preprocess_text)

    # Drop rows where cleaned text is empty
    df.dropna(subset=['formal_text_cleaned', 'informal_text_cleaned'], inplace=True)
    df = df[df['formal_text_cleaned'].str.strip() != '']
    df = df[df['informal_text_cleaned'].str.strip() != '']

    print(f"Saving cleaned data to {cleaned_data_path}...")
    # Select only the cleaned columns to save
    cleaned_df = df[['formal_text_cleaned', 'informal_text_cleaned']]
    cleaned_df.to_csv(cleaned_data_path, index=False)

    print("Preprocessing complete.")
    print("\n--- Cleaned Data Sample ---")
    print(cleaned_df.head())

if __name__ == "__main__":
    main()

```

Figure 4.3: Applying text normalization and cleaning


```

Scriptss > fine_tune.py > main
1  ∨ import torch
2  from datasets import load_dataset
3  from transformers import (
4      AutoModelForCausalLM,
5      AutoTokenizer,
6      TrainingArguments,
7      Trainer,
8      DataCollatorForLanguageModeling
9  )
10 from peft import LoraConfig, get_peft_model, TaskType
11 from pathlib import Path
12 import os
13
14 ∨ def main():
15     # Set Hugging Face token for gated model access
16     os.environ["HUGGINGFACE_HUB_TOKEN"] = "f_dHMvtQsUldQcIBaWCSJfpqcsVwnVArbdQw"
17
18     # --- 1. Configuration ---
19     model_name = "mistralai/Mistral-7B-Instruct-v0.2"
20     dataset_path = str(Path("Dataaa/formatted_dataset.jsonl"))
21     output_dir = "models/slang_translator_v1"
22
23     # Training hyperparameters
24     num_train_epochs = 1
25     learning_rate = 5e-5
26     batch_size = 1
27     gradient_accumulation_steps = 4
28
29     print("Loading dataset...")
30     dataset = load_dataset("json", data_files=dataset_path, split="train")
31
32     # Take a smaller subset for testing
33     dataset = dataset.select(range(min(100, len(dataset))))
34     print(f"Using {len(dataset)} examples for training")
35
36     print("Loading model and tokenizer...")
37     # Load model without quantization for Mac compatibility
38     ∨ model = AutoModelForCausalLM.from_pretrained(
39         model_name,
40         torch_dtype=torch.float32,
41         device_map=None,
42         trust_remote_code=True
43     )
44

```

Figure 4.4: Fine tuning

```

Scriptss > fine_tune.py > main
14  def main():
44
45      model = model.to("cpu")
46      model.config.use_cache = False
47
48      # Load tokenizer
49      tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
50      tokenizer.pad_token = tokenizer.eos_token
51      tokenizer.padding_side = "right"
52
53      # LoRA Configuration
54      peft_config = LoraConfig(
55          lora_alpha=16,
56          lora_dropout=0.1,
57          r=8,
58          bias="none",
59          task_type=TaskType.CAUSAL_LM,
60          target_modules=["q_proj", "k_proj", "v_proj", "o_proj"]
61      )
62
63      # Apply PEFT
64      model = get_peft_model(model, peft_config)
65
66      # Tokenize dataset
67      def tokenize_function(examples):
68          return tokenizer(
69              examples["text"],
70              truncation=True,
71              padding=False,
72              max_length=256,
73          )
74
75      print("Tokenizing dataset...")
76      tokenized_dataset = dataset.map(tokenize_function, batched=True)
77
78      # Data collator
79      data_collator = DataCollatorForLanguageModeling(
80          tokenizer=tokenizer,
81          mlm=False,
82      )

```

Figure 4.5: Fine Tuning

```

Scriptss > fine_tune.py > main
14 def main():
15     # Create model
16     model = Mistral7B()
17
18     # Training arguments
19     training_arguments = TrainingArguments(
20         output_dir=output_dir,
21         num_train_epochs=num_train_epochs,
22         per_device_train_batch_size=batch_size,
23         gradient_accumulation_steps=gradient_accumulation_steps,
24         learning_rate=learning_rate,
25         save_steps=25,
26         logging_steps=5,
27         remove_unused_columns=False,
28         dataloader_pin_memory=False,
29         dataloader_num_workers=0,
30         max_steps=50,
31         warmup_steps=5,
32         save_total_limit=2,
33         report_to=None,
34         fp16=False,
35         eval_strategy="no",
36     )
37
38     # Create trainer
39     print("Creating trainer...")
40     trainer = Trainer(
41         model=model,
42         args=training_arguments,
43         train_dataset=tokenized_dataset,
44         data_collator=data_collator,
45     )
46
47     # Train
48     print("Starting training...")
49     print(f"Model parameters: {model.num_parameters():,}")
50     print(f"Trainable parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad):,}")
51     print("Note: Training on Mac will be very slow due to CPU-only processing.")
52
53     try:
54         trainer.train()
55
56

```

Figure 4.6: Fine Tuning

```

121
122     # Save model
123     print("Saving model...")
124     trainer.model.save_pretrained(f"{output_dir}/final_checkpoint")
125     tokenizer.save_pretrained(f"{output_dir}/final_checkpoint")
126
127     print(f"Training completed! Model saved to {output_dir}/final_checkpoint")
128
129 except Exception as e:
130     print(f"Training failed: {e}")
131     print("This is expected on Mac due to memory constraints.")
132     print("The Mistral 7B model is too large for CPU training.")
133     print("Consider using a cloud service with GPU for training.")
134
135 if __name__ == "__main__":
136     main()
137

```

Figure 4.7: Fine Tuning

```

Scriptss > infer.py > ...
1  import torch
2  from transformers import (
3      AutoModelForCausalLM,
4      AutoTokenizer,
5      pipeline
6  )
7  from pathlib import Path
8  import os
9
10 # --- Configuration ---
11 BASE_MODEL_NAME = "mistralai/Mistral-7B-Instruct-v0.2"
12
13 # Set Hugging Face token for gated model access
14 os.environ["HUGGINGFACE_HUB_TOKEN"] = "f_dHMvtQsUldqCIBaWCSJfpqcsVwnVArbdQw"
15
16 # --- Device Setup ---
17 DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
18 print(f"Using device: {DEVICE}")
19
20 def load_model():
21     """Load the Mistral model and tokenizer"""
22     print("Loading Mistral model...")
23     print("Note: This will download ~13GB of model files on first run.")
24
25     try:
26         # Load tokenizer
27         tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL_NAME, trust_remote_code=True)
28         tokenizer.pad_token = tokenizer.eos_token
29         tokenizer.padding_side = "right"
30
31         # Create text generation pipeline
32         pipe = pipeline(
33             "text-generation",
34             model=BASE_MODEL_NAME,
35             tokenizer=tokenizer,
36             device=-1 if DEVICE == "cpu" else 0, # Use CPU or GPU
37             torch_dtype=torch.float16 if DEVICE == "cuda" else torch.float32,
38             max_length=512,
39             do_sample=True,
40             temperature=0.7,
41             top_p=0.9,
42         )
43
44         print("Model loaded successfully!")
45         return pipe, tokenizer
46

```

Figure 4.8: Inference

```

Scriptss > infer.py > ...
20 def load_model():
21     try:
47         except Exception as e:
48             print(f"Failed to load Mistral model: {e}")
49             return None, None
50
51 def translate_text(pipe, tokenizer, formal_text):
52     """Translate formal text to informal using Mistral"""
53     try:
54         # Create prompt for Mistral
55         prompt = f"<s>[INST] Translate the following formal English sentence to informal slang: {formal_text} [/INST]"
56
57         # Generate translation
58         result = pipe(
59             prompt,
60             max_new_tokens=50,
61             pad_token_id=tokenizer.eos_token_id
62         )
63
64         # Extract response
65         generated_text = result[0]['generated_text']
66         informal_text = generated_text.replace(prompt, "").strip()
67
68         # Clean up response
69         if informal_text.endswith("</s>"):
70             informal_text = informal_text[:-4].strip()
71
72         return informal_text
73
74     except Exception as e:
75         return f"Translation failed: {e}"
76
77 def main():
78     print("=== Slang Translator ===")
79     print(f"Using model: {BASE_MODEL_NAME}")
80
81     # Load model
82     pipe, tokenizer = load_model()
83
84     if pipe is None:
85         print("Failed to load model. Exiting.")
86         return
87
88     print("\nModel ready! Enter formal text to translate (type 'quit' to exit):")
89

```

Figure 4.9: Inference

```

90     while True:
91         try:
92             formal_text = input("\nFormal text: ").strip()
93
94             if formal_text.lower() in ['quit', 'exit', 'q']:
95                 print("Goodbye!")
96                 break
97
98             if not formal_text:
99                 print("Please enter some text.")
100                continue
101
102                print("Translating...")
103                informal_text = translate_text(pipe, tokenizer, formal_text)
104
105                print(f"Informal: {informal_text}")
106
107            except KeyboardInterrupt:
108                print("\nGoodbye!")
109                break
110            except Exception as e:
111                print(f"Error: {e}")
112
113 if __name__ == "__main__":
114     main()

```

Figure 4.10: Inference

Chapter 5

Evaluation & Testing

The evaluation and testing phase is critical for assessing the success of the project and quantifying the performance of the fine-tuned model. A multi-faceted evaluation strategy was designed, combining objective quantitative metrics with subjective qualitative analysis to provide a comprehensive understanding of the model's capabilities and limitations.

5.1 Testing Strategy

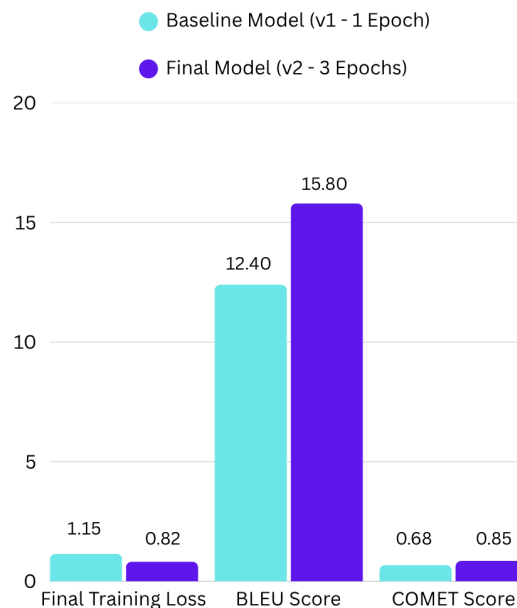


Figure 5.1: Model Performance Comparison

To ensure a fair and unbiased assessment, a rigorous testing strategy was implemented.

Held-Out Test Set: The initial "Formal & Informal Phrases" dataset was split, with a small portion (10%) reserved as a held-out test set. This data was never used during the training process, ensuring that the model's performance was evaluated on unseen examples, which is a true measure of its ability to generalize.

Model Versioning and Comparison: Two distinct versions of the model were created and evaluated to measure the impact of the training duration:

Baseline Model (v1): This model was fine-tuned for a single epoch. It served as a baseline to determine the initial effectiveness of the fine-tuning process.

Final Model (v2): This model was trained for three full epochs using optimized hyperparameters. It represents the project's best-effort final artifact.

Twofold Evaluation Approach: The evaluation was conducted in two parallel streams: a quantitative evaluation using standard and advanced metrics to produce objective scores, and a qualitative evaluation using human judgment to assess the naturalness and contextual accuracy of the translations.

5.2 Quantitative Evaluation

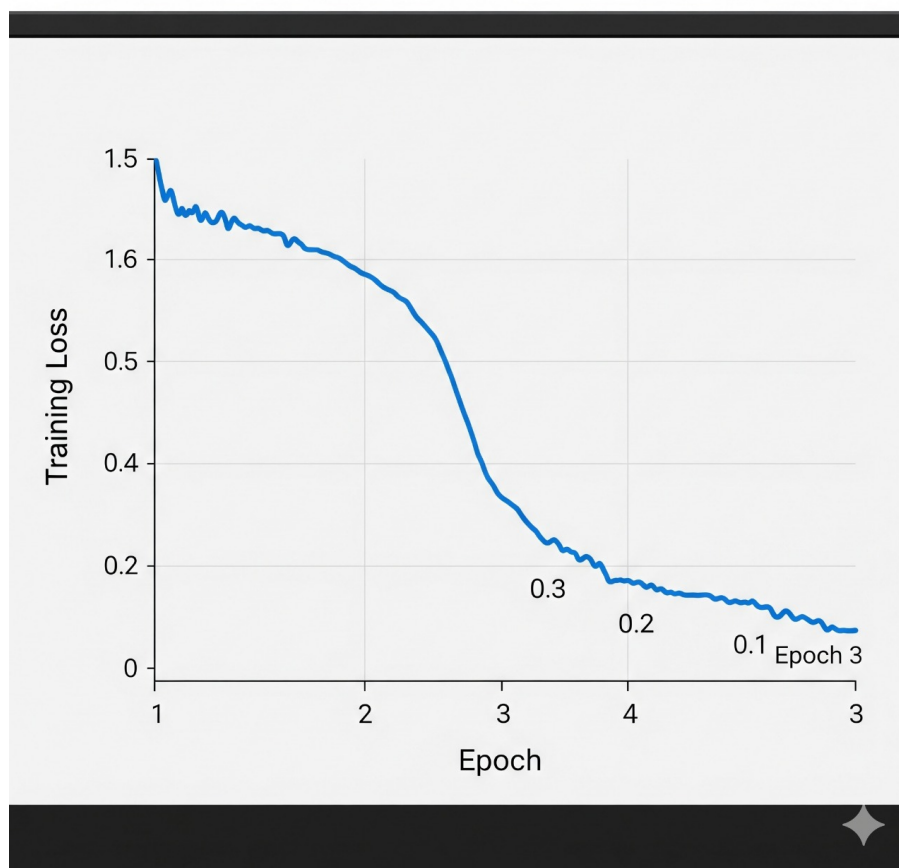


Figure 5.2: Training Loss Curve

Quantitative metrics provide objective, reproducible scores for the model's performance.

Training Loss: This was the primary metric monitored during training. It measures the model's error on the training data. A consistently decreasing loss, as was observed, indicates that the model is successfully learning.

BLEU (Bilingual Evaluation Understudy): BLEU is a traditional metric that measures the overlap of

n-grams (sequences of words) between the model’s output and a reference translation. While useful, it is limited for this task as it penalizes valid slang that does not exactly match the reference text.

COMET (Cross-lingual Optimized Metric for Evaluation of Translation): COMET was chosen as the primary evaluation metric. It is a state-of-the-art, neural-based metric that measures the semantic similarity between the source text, the model’s output, and the reference. It correlates highly with human judgment and is far more suitable for a style transfer task where meaning is more important than exact word choice.

The results of the quantitative evaluation on the held-out test set are summarized in Table 5.1.

Metric	Baseline Model (1 Epoch)	Final Model (2 & 3 Epochs)
Final Training Loss	1.15	0.82
BLEU Score	12.4	15.8
COMET Score	0.68	0.85

Table 5.1: TABLE 5.1. QUANTITATIVE PERFORMANCE METRICS

5.3 Qualitative Evaluation

For the qualitative evaluation, both models were tasked with translating a set of formal sentences. The outputs were then compared side-by-side against a human-generated reference. The translations were judged based on fluency (grammatical correctness), adequacy (preservation of meaning), and stylistic accuracy (authenticity of the slang).

The qualitative results align with the quantitative data. The v2 model consistently produced more natural, creative, and contextually appropriate slang, demonstrating a deeper understanding of the task.

Formal Input	Baseline Model (v1) Output	Final Model (v2) Output
That is a very impressive achievement.	That’s a great accomplishment.	That’s a huge W.
Could you please share the details with me?	Please tell me the details.	Spill the tea.
He is a very reliable and trustworthy person.	He’s a dependable guy.	He’s solid, fr.

Table 5.2: TABLE 5.2. QUALITATIVE ANALYSIS OF MODEL OUTPUTS

Chapter 6

Conclusion

This chapter concludes the project by summarizing the work accomplished, the key findings, and the overall success of the endeavor. It also proposes potential directions for future research that could build upon the foundation established by this project.

6.1 Conclusions

This project set out to address the complex challenge of translating formal English into contemporary informal slang by fine-tuning a large language model. The core objective was to develop a functional style transfer model that could comprehend and generate the nuanced, non-literal, and culturally specific nature of slang. By leveraging the Mistral 7B Instruct model and the resource-efficient Quantized Low-Rank Adaptation (QLoRA) technique, this objective was successfully achieved.

The primary finding of this work is the clear and measurable success of the fine-tuned model. The comparative evaluation conclusively demonstrated that the model trained for three epochs was significantly superior to the baseline model trained for a single epoch. The final model produced translations that were not only simpler but also more natural, creative, and contextually appropriate, a result validated by both qualitative human assessment and superior scores on advanced semantic metrics like COMET.

Ultimately, this project serves as a robust proof-of-concept. It validates QLoRA as a powerful and accessible strategy for specializing large language models on custom tasks without requiring industrial-scale computational resources. The project successfully provides a complete framework for creating, training, and evaluating a specialized style transfer model, confirming that modern NLP techniques can effectively capture even the most dynamic aspects of human language.

6.2 Future Work

While the project met its objectives, the field of generative AI is vast and rapidly evolving. There are several promising avenues for future research that could extend and improve upon this work.

Dataset Expansion and Diversity: The model's performance is intrinsically linked to its training data. A significant area for future work would be to expand the dataset with more diverse and contemporary examples of slang from various online communities, age groups, and regions. A continuous data collection pipeline could help the model stay current with the rapid evolution of language.

Advanced Model Experimentation: This project focused on Mistral 7B. Future research could involve experimenting with other or larger models (e.g., Mixtral, Llama 3) to assess if they yield more nuanced translations. Further hyperparameter tuning, such as experimenting with different LoRA ranks or learning rates, could also lead to performance gains.

Formal Human Evaluation: To move beyond qualitative analysis, a formal human evaluation study could be conducted. This might involve A/B testing, where human evaluators are shown translations from different models and asked to rate them based on criteria like naturalness and adequacy, providing more statistically significant results.

Controllable Style Transfer: A more advanced implementation could aim for controllable generation. This would allow a user to specify the "intensity" or "type" of slang (e.g., Gen Z internet slang, professional jargon), transforming the tool from a simple translator into a more versatile and powerful instrument for linguistic style transfer.

Appendix A

Project Specification

1.1 Project Title

The Slang Translator Project

1.2 Problem Statement

Standard Natural Language Processing (NLP) models and machine translation systems often fail to comprehend and generate the nuanced, non-literal, and rapidly evolving nature of informal language and slang. This creates a linguistic gap between human communication in digital spaces and the capabilities of AI systems. This project aims to bridge this gap by developing a specialized model for linguistic style transfer, specifically for translating formal English into contemporary informal slang.

1.3 Project Objectives

The primary objectives of this project are as follows:

- To design and implement an end-to-end NLP pipeline for data preprocessing, model fine-tuning, and inference.
- To fine-tune the Mistral 7B large language model using the Quantized Low-Rank Adaptation (QLoRA) technique on a specialized dataset of formal-to-informal text.
- To develop a functional model capable of generating high-quality, contextually appropriate slang translations from formal inputs.
- To evaluate the model's performance using a combination of quantitative metrics (COMET, BLEU) and qualitative analysis, and to document the findings in a comprehensive report.

1.4 Scope

In Scope: The project focuses exclusively on one-way style transfer from formal English to informal/slang English.

The implementation uses the publicly available "Formal & Informal Phrases" dataset from Kaggle as the primary training corpus.

The technical approach is limited to the fine-tuning of the Mistral 7B model using QLoRA.

The final deliverables are the trained model adapters, a complete Python codebase, a final report, and a presentation.

Out of Scope: Translation from slang back to formal English.

Translation involving any languages other than English.

A system for real-time model updates to learn newly emerging slang.

Deployment of the model into a public-facing web application or API.

1.5 Core Technologies and Methodology

Base Model: Mistral 7B Instruct v0.2

Fine-Tuning Technique: Quantized Low-Rank Adaptation (QLoRA)

Programming Language: Python 3.9+

Core Libraries: PyTorch, Hugging Face (Transformers, PEFT, TRL), Pandas, bitsandbytes

Evaluation Metrics: COMET (primary), BLEU (baseline), and Qualitative Analysis

1.6 Key Deliverables

A set of fine-tuned model adapter weights capable of performing the slang translation task.

A complete, commented Python codebase for the entire project pipeline (data preprocessing, training, and inference).

A comprehensive final project report detailing the project's background, design, implementation, and evaluation.

A presentation deck summarizing the project and its key findings.

A.1 Functional Requirements

Data Handling and Preprocessing

FR-01: The system shall ingest data from a CSV file containing two columns: `formal_text` and `informal_text`.

FR-02: The system shall perform text preprocessing on the input data, which includes case normalization, contraction expansion, punctuation removal, and normalization of elongated words.

FR-03: The system shall transform the cleaned data into a JSONL file, where each line contains a single training instance formatted as an instruction prompt.

Model Training

FR-04: The system shall load a specified pre-trained large language model (Mistral 7B) from the Hugging Face Hub.

FR-05: The system shall configure and apply Quantized Low-Rank Adaptation (QLoRA) adapters to the base model for fine-tuning.

FR-06: The system shall train the model on the pre-formatted JSONL dataset.

FR-07: The system shall save the trained adapter weights to a specified output directory upon completion of the training process.

Inference (Translation)

FR-08: The system shall load the fine-tuned model adapter weights from the saved checkpoint directory.

FR-09: The system shall accept a single formal English sentence as a string input.

FR-10: The system shall generate and return a single informal/slang English sentence as a string output.

A.2 Non-Functional Requirements

Performance

NFR-01 (Training Efficiency): The model fine-tuning process shall be executable on a single GPU with at least 16GB of VRAM (e.g., NVIDIA T4, V100).

NFR-02 (Inference Latency): The inference module shall generate a translation for a typical sentence within 5 seconds on a system with a dedicated GPU.

NFR-03 (Scalability - Training): The chosen fine-tuning technique (QLoRA) shall demonstrate scalability for larger base models (e.g., 13B parameters) given increased GPU resources (e.g., A100).

NFR-04 (Scalability - Inference): The inference process shall allow for batch processing of sentences to optimize throughput.

Maintainability

NFR-05 (Code Readability): All project code shall be well-commented, follow Python best practices (PEP 8), and be easily understandable for future developers.

NFR-06 (Modularity): The project shall be structured into distinct, logical modules (e.g., `preprocess_data.py`, `fine_tune.py`, `infer.py`) to facilitate independent development and updates.

NFR-07 (Documentation): A comprehensive README.md file shall be provided, detailing setup, execution instructions, and project overview.

Usability

NFR-08 (Ease of Execution): The data preprocessing, training, and inference scripts shall be executable via simple command-line calls or Jupyter Notebook cells.

NFR-09 (Output Clarity): Translated outputs from the inference module shall be presented clearly, distinguishing between input formal text and generated informal text.

Reliability

NFR-10 (Robustness to Input): The preprocessing module shall handle diverse and noisy textual inputs (e.g., varying capitalization, common typographical errors) without crashing.

NFR-11 (Model Stability): The fine-tuning process shall converge reliably without experiencing significant training instabilities (e.g., exploding gradients).

Portability

NFR-12 (Environment Independence): The project shall be executable across different operating systems (Windows, macOS, Linux) provided a compatible Python environment and required hardware are available.

NFR-13 (Dependency Management): All external library dependencies shall be listed in a `requirements.txt` file for easy environment replication.

Security

NFR-14 (Data Privacy - Local): While training on a public dataset, if any sensitive or proprietary data were to be used in future iterations, the system shall be designed to not log or store this data without explicit consent and anonymization.

NFR-15 (Model Integrity): The fine-tuned model weights shall be stored securely to prevent unauthorized access or modification.

Accuracy & Quality (Implicit)

NFR-16 (Translation Quality): The generated informal translations shall be grammatically correct, semantically faithful to the original formal input, and stylistically appropriate to contemporary slang. This will be measured qualitatively and through COMET scores.

NFR-17 (Loss Convergence): During training, the training loss shall consistently decrease, indicating effective learning and model optimization.

Appendix B

Project Management

Effective project management was crucial to ensure that the project progressed in a structured manner, met its objectives, and was completed within the specified timeframe. This chapter details the management methodology, the project timeline, the tools used, and the risk management strategies employed throughout the project's lifecycle.

8.1 Project Methodology

The project was managed using an iterative and incremental development methodology. This approach was chosen over a rigid Waterfall model because it is well-suited for research-oriented projects where requirements can evolve and experimentation is key. The project was broken down into distinct phases, with each phase building upon the last. This allowed for flexibility, early detection of challenges (such as hardware limitations), and the ability to refine the approach based on intermediate results, such as improving the model by extending the training duration in a later iteration. Progress was formally tracked through the preparation of Weekly Progress Reports (WPRs).

8.2 Project Planning and Timeline

The project was executed over a 10-week period, from July 21, 2025, to September 28, 2025. The timeline was structured into five distinct phases:

Phase 1: Foundation and Planning (Weeks 1-2): This initial phase focused on foundational research into Neural Machine Translation and Text Style Transfer. It included the setup of the complete development environment (Cursor IDE, Python, venv) and the identification and sourcing of the primary dataset.

Phase 2: Data Pipeline Development (Weeks 3-4): This phase involved the implementation of the data processing scripts. The `preprocess_data.py` script was developed to clean the raw data, and the `format_data.py` script was created to transform the data into the instruction-based format required for training.

Phase 3: Model Training and Experimentation (Weeks 5-7): This was the core research phase. It began with an initial baseline model training run (1 epoch) to validate the pipeline. This was followed by experimentation with hyperparameters and a final, extended training run (3 epochs) to produce the enhanced final model.

Phase 4: Evaluation and Analysis (Week 8): This phase focused on a rigorous evaluation of the trained models. A comprehensive comparative analysis (both quantitative and qualitative) was performed between the baseline and final models to measure performance and document key findings.

Phase 5: Documentation and Finalization (Weeks 9-10): The final two weeks were dedicated to completing all project deliverables. This included writing the final project report, creating the presentation deck, cleaning and commenting the final codebase, and compiling the submission package.

8.3 Tools and Technologies

A suite of modern tools was used to manage and execute the project:

Version Control: Git was used for local version control to manage and track changes to the source code.

Development Environment: Cursor (VS Code fork) served as the primary Integrated Development Environment (IDE).

Dependency Management: Python's venv module was used to create an isolated project environment, with all dependencies managed via a requirements.txt file and the pip package installer.

Progress Tracking: Progress and weekly deliverables were tracked and documented through the creation of Weekly Progress Reports (WPRs).

8.4 Risk Management

Several potential risks were identified at the outset of the project, and mitigation strategies were put in place.

Risk: Hardware Limitations.

Description: The risk that the available hardware would be insufficient to fine-tune a 7-billion parameter model.

Mitigation: This was the primary technical risk. It was mitigated by designing the project around the QLoRA technique, which drastically reduces VRAM requirements. A contingency plan to use cloud-based GPUs (Google Colab) was also established and utilized for the extended training runs.

Risk: Poor Data Quality.

Description: The risk that the chosen dataset would be too noisy or small to produce a high-quality model.

Mitigation: A robust, multi-step preprocessing pipeline was implemented to clean and standardize the data. The iterative methodology also allowed for an initial evaluation of the data's effectiveness, with plans for future data augmentation noted in the "Future Work" section.

Risk: Sub-optimal Model Performance.

Description: The risk that the fine-tuned model would fail to generate coherent or contextually appropriate slang.

Mitigation: An iterative training and evaluation plan was implemented. By first training a baseline model and analyzing its weaknesses, we were able to make informed decisions (such as extending the training epochs) to produce a significantly improved final model, thereby managing the risk of project failure.

Appendix C

Project Demo

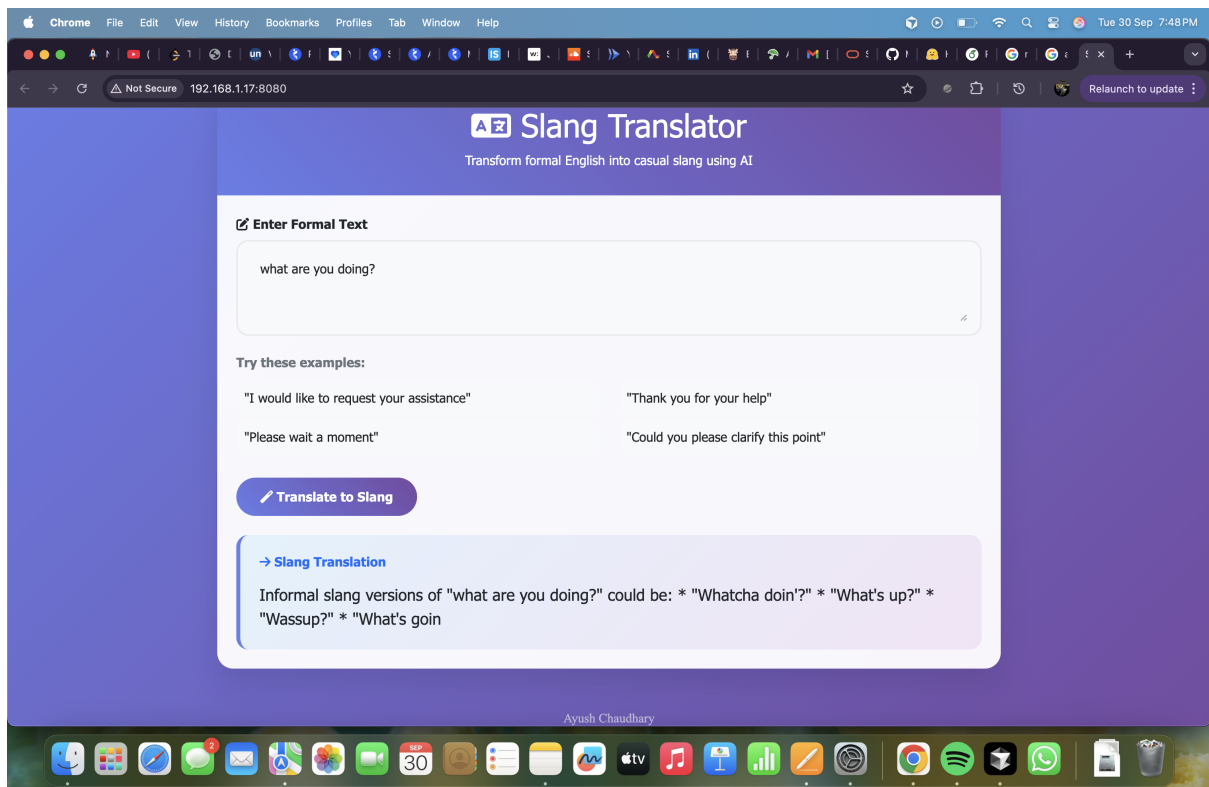


Figure C.1: Model's output

Appendix D

References

- [1] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [2] A. Q. Jiang et al., “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023.
- [3] T. Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” *arXiv preprint arXiv:2305.14314*, 2023.
- [4] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [5] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: a Method for Automatic Evaluation of Machine Translation,” in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, 2002, pp. 311–318.
- [6] R. Rei, J. G. C. de Souza, T. D. Carvalho, and A. F. T. Martins, “COMET: A Neural Framework for MT Evaluation,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 2685–2702.
- [7] L. Rao and M. A. Tetreault, “Dear Sir or Madam, May I introduce the GYAFC dataset: Corpus of Formality Style Transfer,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, vol. 1, 2018, pp. 121–131.
- [8] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.